

Parte VI

DOM-JAVA SCRIPT

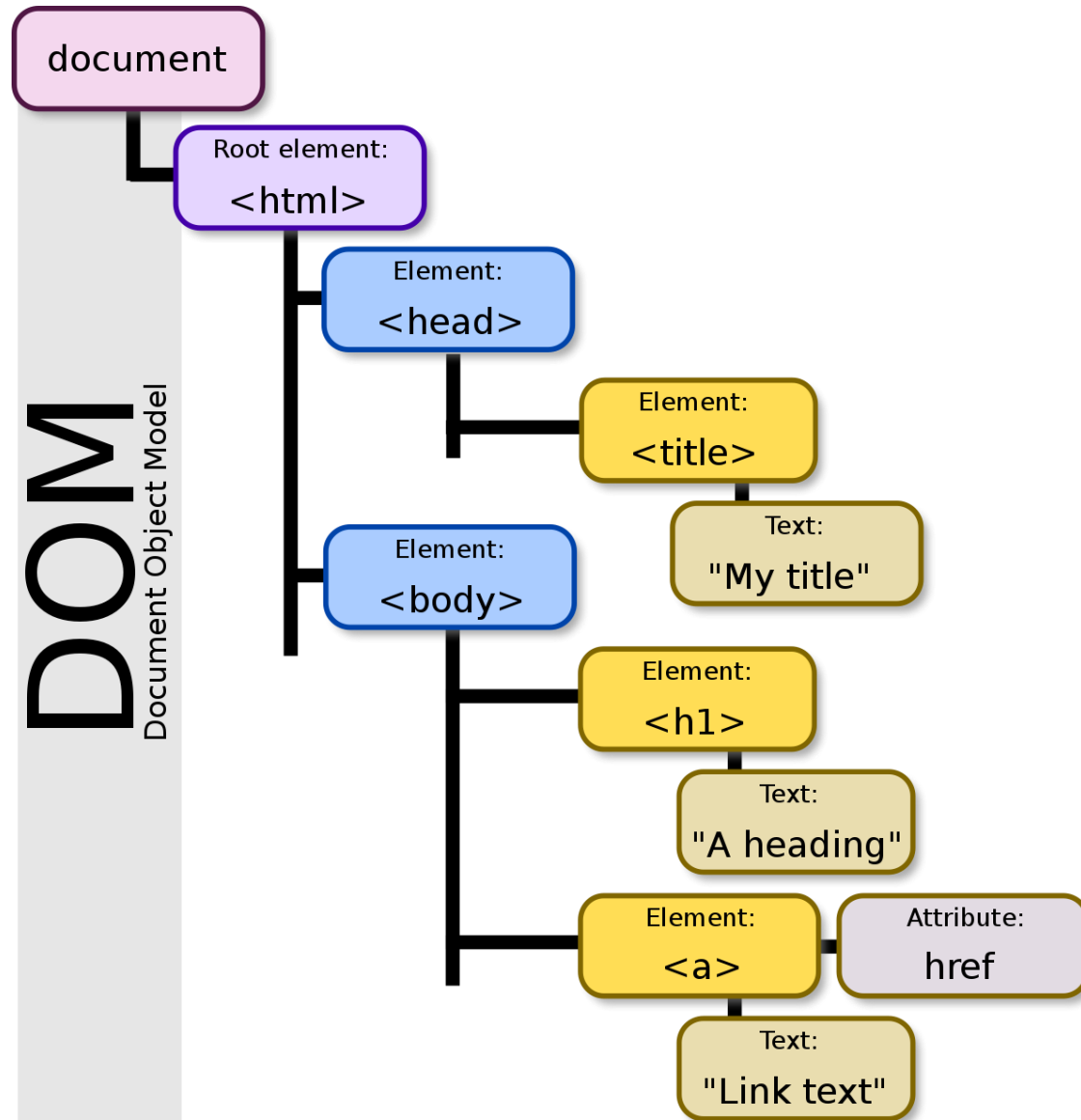




¿Qué es DOM (Document Object Model)?

El navegador
convierte el
HTML en un árbol

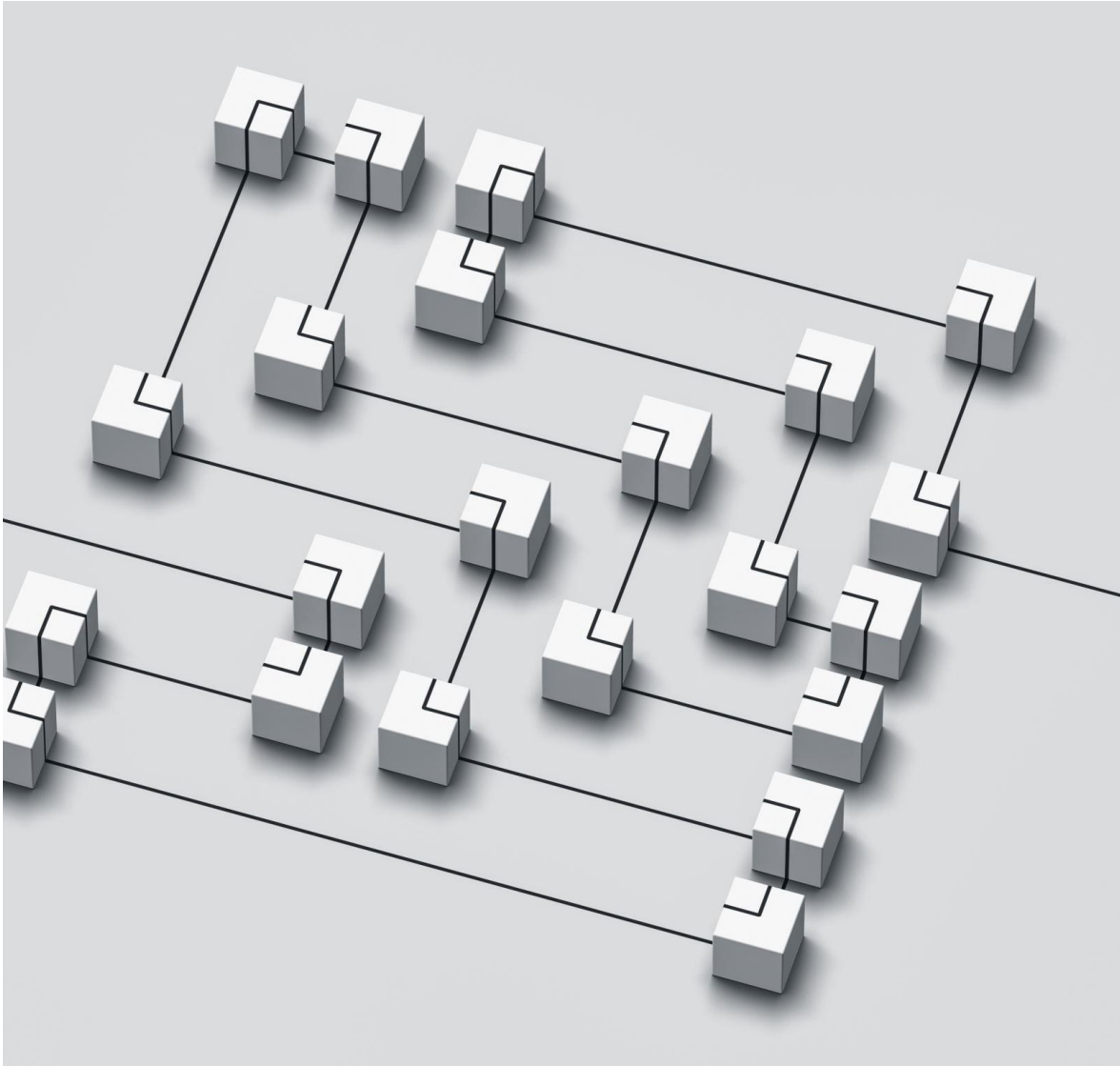
Cada etiqueta =
un nodo



¿Por qué árbol?

```
<html>
  <body>
    <h1>Título</h1>
    <p>Párrafo</p>
  </body>
</html>
```

```
html
├── body
│   ├── h1
│   └── p
```



¿Qué es un nodo?

- Un nodo es cada elemento que forma parte del DOM
- Como funciona realmente
 1. Escribes un HTML
 2. El navegador lo carga
 3. El navegador crea el DOM
 4. JavaScript trabaja sobre ese DOM



document

document es el objeto que representa toda la página web cargada en el navegador.

Escribe en la consola del navegador document, estas accediendo a toda la página

getElementById

- Busca un elemento por su atributo id.
- Ejemplo:

<h1 id="titulo">Hola clase</h1>

Aquí hay un h1.
Tiene un atributo llamado id.
Su valor es "titulo".

document.getElementById("titulo")

document: entra en la página
getElementById(...): busca un
elemento por su id
"titulo": el id que quiero buscar



querySelector

- Busca el **primer elemento** que coincida con un selector CSS.
- Ejemplo

`document.querySelector("#titulo");`

Esto busca el elemento con id titulo.

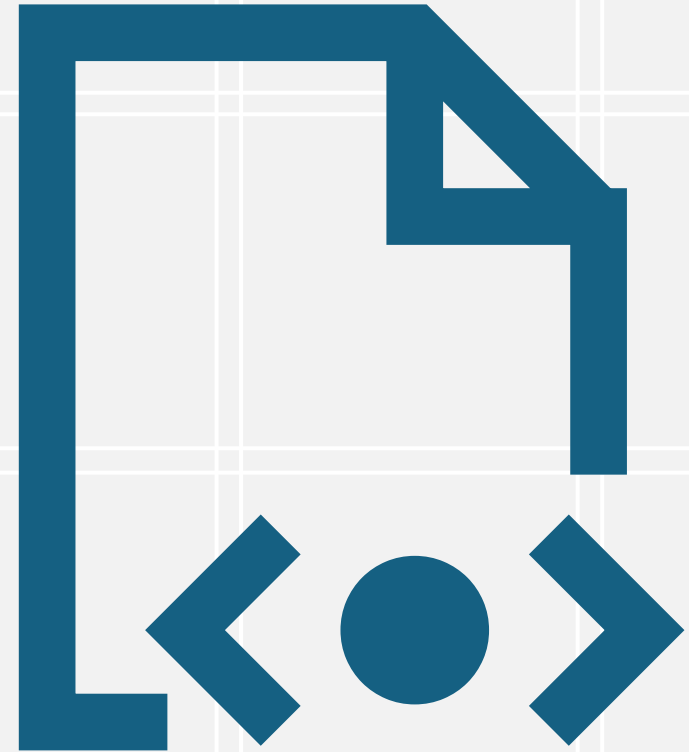
RECORDATORIO CSS

`#titulo` significa “el elemento con id titulo”

`.caja` significa “los elementos con clase caja”

`P` significa “las etiquetas p”

```
document.querySelector("#titulo");  
document.querySelector("p");  
document.querySelector(".rojo");
```



Diferencias

getElementById("titulo")

Busca por id

Solo sirve para ids

No lleva #

querySelector("#titulo")

usa sintaxis CSS

sirve para id, clase, etiqueta...

para id sí lleva #

Típico error

- `document.querySelector("titulo");`

Eso buscaría una etiqueta `<titulo>`, no un id.



querySelectorAll

- No devuelve un solo elemento.
- Devuelve una **lista de elementos**.
- Eso es importantísimo.
- Si hay tres párrafos, devuelve los tres.

```
<p>Uno</p>  
<p>Dos</p>  
<p>Tres</p>
```

```
let parrafos = document.querySelectorAll("p");  
console.log(parrafos);
```

Ejemplo practico

- ¿Qué devuelve `getElementById`?
- ¿Qué busca `querySelector("p")`?
- ¿Qué devuelve `querySelectorAll("p")`?
- ¿Cuál es la diferencia entre uno y otro?

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Práctica</title>
</head>
<body>
  <h1 id="titulo">Bienvenidos</h1>
  <p>Texto 1</p>
  <p>Texto 2</p>
  <p>Texto 3</p>

  <script src="script.js"></script>
</body>
</html>
```

</> JavaScript

```
let titulo = document.getElementById("titulo");
console.log(titulo);

let primerParrafo = document.querySelector("p");
console.log(primerParrafo);

let parrafos = document.querySelectorAll("p");
console.log(parrafos);
```

Soluciones

- 1 `<h1 id="titulo">Bienvenidos</h1>` Devuelve el h1 completo
- 2 `let primerParrafo = document.querySelector("p");` Busca el primer `<p>` que aparece em HTML
- 3 `let parrafos = document.querySelectorAll("p");` Devuelve la lista con:

```
<p>Texto 1</p>  
<p>Texto 2</p>  
<p>Texto 3</p>
```



Ejercicio 1

- Crea una página con:
 - un h1 con id
 - dos párrafos
 - un botón
 - Y en JavaScript:
 - selecciona el h1
 - selecciona el botón
 - selecciona todos los párrafos
 - muestra todo por consola
-

Métodos

Cambiar contenido

- `elemento.textContent = "Nuevo texto";` Cambia el texto

Cambiar HTML

- `elemento.innerHTML = "<b>Hola</b>";` Permite meter HTML dentro

Cambiar estilos

- `elemento.style.color = "red";` Cambia el CSS directamente

Añadir clases

- `elemento.classList.add("activo");`
- `elemento.classList.remove("activo");`

Crear elementos

- `let nuevo = document.createElement("p");`
`nuevo.textContent = "Nuevo párrafo";`

Añadir al documento

- `document.body.appendChild(nuevo);` Lo añade al final del body

Eliminar elementos

- `elemento.remove();`
- **Añadir un elemento dentro de otro**
- `lista.appendChild(li);`

Nodos

1. Padre (parent)

- `elemento.parentNode` da el elemento “de arriba”

2. Hijos (children)

- `elemento.children` .Todos los hijos (HTMLCollection)
- `elemento.firstElementChild` . Primer hijo
- `elemento.lastElementChild`. Último hijo

3. Hermanos (siblings)

- `elemento.nextElementSibling` .Siguiete hermano
- `elemento.previousElementSibling`. Hermano anterior

Ejemplo Practico

Vamos a crear una lista de tareas que funcione de verdad

- **PASO 1: Estructura básica**

- Un título
- Un input para escribir
- Un botón
- Una lista vacía (ul/li)

- **PASO 2: Conectar con JavaScript**

Ahora tenemos que poder acceder a esos elementos desde JavaScript

“¿Cómo seleccionamos elementos del DOM?” getElementById / querySelector

- **PASO 3: Detectar el click**

Queremos que pase algo cuando pulsemos el botón

- “¿Qué usamos para escuchar eventos?” addEventListener
- “¿Qué evento?” click

PASO 4: Obtener lo que escribe el usuario

Cuando hacemos click, necesitamos coger el texto del input

- “¿Dónde está ese valor?” → .value

Ejemplo Practico

- **PASO 5: Crear una nueva tarea**

Ahora tenemos que crear un nuevo elemento en la página

- “¿Qué etiqueta usamos para una tarea?” → li
- “¿Cómo se crea un elemento desde JS?” createElement

- **PASO 6: Meter el texto dentro**

Ese elemento tiene que tener el texto que ha escrito el usuario
→ `textContent`

- **PASO 7: Añadirlo a la lista**

- Ya hemos creado la tarea... pero no se ve
- “¿Qué nos falta para que aparezca en pantalla?” → `appendChild`

- **PASO 8: Limpiar el input**

- Después de añadir la tarea... queremos borrar lo que había escrito

Ejemplo Practico

- **PASO 9: Marcar como completada**

¿Y si quiero tachar una tarea cuando hago click sobre ella?

- Guía: evento sobre el li ,cambiar estilo

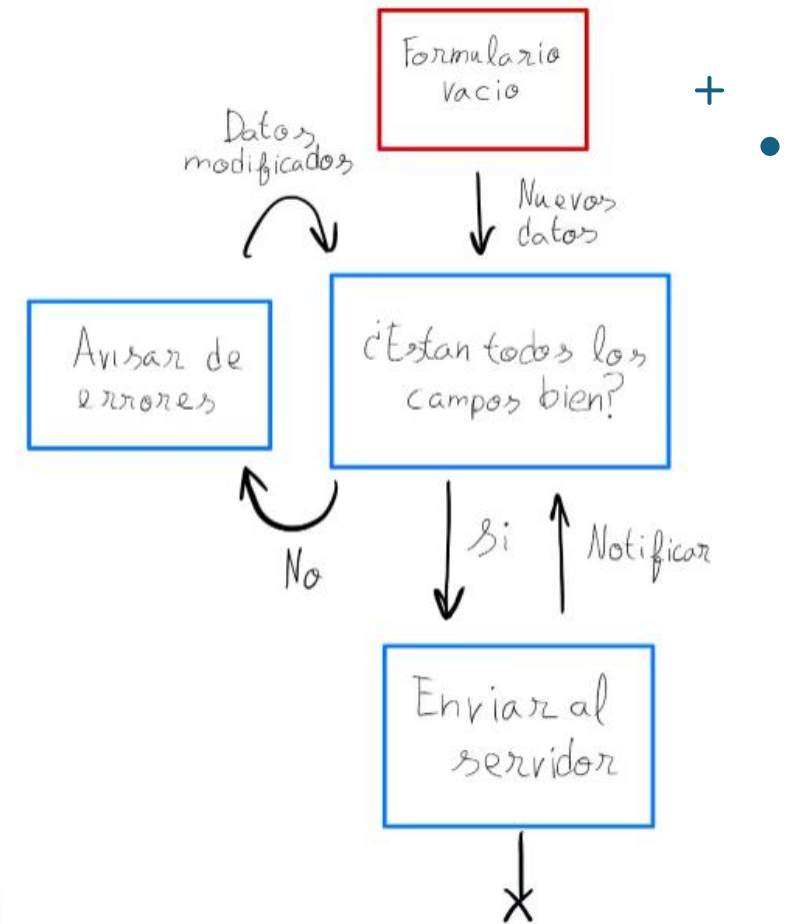
- **PASO 10: Eliminar tarea**

¿Cómo podríamos borrar una tarea?

- Pistas:doble click o botón eliminar

Formularios

- **Reposo:** El usuario aún no ha introducido información. Los campos están vacíos, vírgenes, esperando ser rellenos.
- **Validación:** Se incluye los datos. Los campos deben cumplir unas directrices. En caso contrario se muestra al usuario como debe corregirlo.
- **Envío:** Los campos cumple las necesidades descritas. Se envía al servidor o servicio (como un API).
- **Respuesta del servidor:** Se informa al usuario de si ha llegado correctamente o ha fallado.



¿Cómo harías el código?

Nombre:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Formulario</title>
</head>
<body>

  <form>
    <label for="name">Nombre: </label>
    <input type="text" name="name" id="name" required autocomplete="off">
    <button type="submit">Enviar</button>
  </form>

  <script>
    let formulario = document.querySelector("form");
    let input = document.getElementById("name");

    formulario.addEventListener("submit", function(event) {

      event.preventDefault(); // evita que se recargue la página

      let nombre = input.value;

      alert("Hola " + nombre);

    });
  </script>

</body>
</html>
```

Mensajes personalizados

Para personalizar el mensaje, como quitarlo cuando esté relleno, deberemos jugar con los eventos input (escribe dentro del campo) como invalid (no se cumple la validación del campo) que usaremos para mostrar el mensaje deseado con `setCustomValidity()`.

Es recomendable usar `autocomplete="off"` para evitar que el autocompletador de los navegadores tapen las validaciones.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Formulario con validación</title>
</head>
<body>

  <form>
    <label for="name">Nombre: </label>
    <input type="text" id="name" required autocomplete="off">
    <button type="submit">Enviar</button>
  </form>

  <script>
    const miInput = document.querySelector('input');

    // Quita el mensaje mientras escribes
    miInput.addEventListener('input', () => {
      miInput.setCustomValidity('');
      miInput.checkValidity();
    });

    // Muestra mensaje personalizado si está vacío
    miInput.addEventListener('invalid', () => {
      miInput.setCustomValidity('Si no es molestia... ¿me dices tu nombre?');
    });
  </script>

</body>
</html>
```


Límite de caracteres

Incluimos el atributo pattern en el input a validar, con una secuencia de patrón regular.

```
<!-- Formulario -->
<form>
  <!-- Etiqueta del input -->
  <label for="marca">Marca:</label>

  <!--
  Input de texto:
  - required: obliga a escribir algo
  - pattern: regla que debe cumplir el texto
  - autocomplete="off": evita sugerencias del navegador
  -->
  <input
    type="text"
    id="marca"
    required
    pattern="^[a-zA-Z0-9]{4,}$"
    autocomplete="off"
  >

  <!--
  EXPLICACIÓN DEL PATTERN:
  ^          -> inicio del texto
  [a-zA-Z0-9] -> solo permite letras minúsculas, mayúsculas y números
  {4,}       -> mínimo 4 caracteres
  $          -> final del texto

  Traducción : "El texto debe tener 4 o más caracteres y solo puede contener letras o números"
  -->
```

```
<!-- Boton para enviar -->
<input type="submit" value="Enviar">
</form>

<script>
  // Seleccionamos el input por su id
  const inputMarca = document.querySelector('#marca');

  // Guardamos el mensaje de error personalizado en una constante
  const mensajeError = "Muy corta. Dame un nombre con 4 o más caracteres.";

  // Evento mientras el usuario escribe
  inputMarca.addEventListener('input', () => {
    // Borra el mensaje de error personalizado
    inputMarca.setCustomValidity('');

    // Comprueba otra vez si ahora el campo es válido
    inputMarca.checkValidity();
  });

  // Evento cuando el campo no pasa la validación
  inputMarca.addEventListener('invalid', () => {
    // Mostramos nuestro mensaje personalizado
    inputMarca.setCustomValidity(mensajeError);
  });
</script>
```

Carácteres válidos

- En este caso comprobaremos 2 casos:
 - Debe estar relleno.
 - Los caracteres permitidos son alfanuméricos, incluyendo espacios.
- Incluimos el atributo pattern en el input a validar, con una secuencia de patrón regular.

Quitar espacios innecesarios

Para evitar que nos envíen campos con espacios al principio y al final de un texto podemos ayudarnos de la función `trim()`.

```
trim('  Time After time  ')
```

Ejemplo:

- blur es un **evento del DOM** que ocurre cuando: **un elemento pierde el foco**. Es el elemento en el que estás escribiendo o interactuando
- Haces click en un input → tiene foco
- Haces click fuera → pierde foco → **blur**

```
// Quita los espacios al principio y al final  
miInput.addEventListener('blur', () => {  
    miInput.value = miInput.value.trim();  
});
```

Ejemplo

Email

```
<form>
  <label for="email">Email: </label>
  <input type="email" name="email" id="email" required autocomplete="of
f">
  <button>Enviar</button>
</form>
```

```
<input type="email" required>
```

```
const miInput = document.querySelector('input');

// Quita la validación mientras escribes
miInput.addEventListener('input', () => {
  // Quita el mensaje según escribes
  miInput.setCustomValidity('');
  // Comprueba si debe validarlo
  miInput.checkValidity();
});

// Muestra el mensaje de validación
miInput.addEventListener('invalid', () => {
  miInput.setCustomValidity('Me parece que esto no es un E-mail');
});
```

Número

```
<input type="number" required>
```

```
<form>
  <label for="email">Email: </label>
  <input type="email" name="email" id="email" required autocomplete="of
f">
  <button>Enviar</button>
</form>
```

```
const miInput = document.querySelector('input');

// Quita la validación mientras escribes
miInput.addEventListener('input', () => {
  // Quita el mensaje según escribes
  miInput.setCustomValidity('');
  // Comprueba si debe validarlo
  miInput.checkValidity();
});

// Muestra el mensaje de validación
miInput.addEventListener('invalid', () => {
  miInput.setCustomValidity('No es un número');
});
```

Checkbox

- Es bastante común **obligar a que un usuario acepte un checkbox** antes de dejarle continuar .Sea el caso que sea, no difiere mucho de lo visto en anterioridad. Haremos uso el atributo required.

☐ Acepto vender mi alma

```
<form>
  <label>
    <input type="checkbox" required autocomplete="off"> Acepto vender
mi alma
  </label>
  <button>Enviar</button>
</form>
```

```
const miInput = document.querySelector('input');

// Quita la validación mientras escribes
miInput.addEventListener('input', () => {
    // Quita el mensaje según escribes
    miInput.setCustomValidity('');
    // Comprueba si debe validarlo
    miInput.checkValidity();
});

// Muestra el mensaje de validación
miInput.addEventListener('invalid', () => {
    miInput.setCustomValidity('Si no aceptas no puedes continuar');
});
```


File

- En el siguiente ejemplo validaremos que el campo de archivo, o tipo file, cumpla los siguientes requisitos:
- Extensiones: jpg, jpeg y png.
- No supere los 2 Mb.

Mi foto: Ningún archivo seleccionado

```
<input type="file" id="foto">
```

```
const oneMegabytesInBytes = 10 ** 6;
const pesoLimite = oneMegabytesInBytes * 2; // 2 megabyte
const extensionesPermitidas = ['jpg', 'jpeg', 'png'];
const miInput = document.querySelector('#foto');

function validarImagen () {
    // Resetea mensaje
    miInput.setCustomValidity('');

    // Deestructuramos para obtener el nombre y el tamaño
    const { name: archivoNombre, size: archivoPeso } = this.files[0];

    // Obtenemos la extensión
    const fileExtension = archivoNombre.split(".").pop();

    // Validamos si tienes una extensión válida
    if (!extensionesPermitidas.includes(fileExtension)){
        miInput.setCustomValidity('Formato no válido, solo se admite jpg y png');
    }

    // Validamos el peso
    if(archivoPeso > pesoLimite) {
        miInput.setCustomValidity('Demasiado grande');
    }
}

miInput.addEventListener("input", validarImagen);
```

Envío

Si todos los campos son correctos se enviará, no obstante, si existe la necesidad de enviarlo sin pulsar el botón haremos uso de la función submit() de DOM formulario

```
<form id="mi-formulario">  
  <input type="text" value="">  
</form>
```

```
const miFormulario = document.querySelector('#mi-formulario');  
  
miFormulario.submit();
```

Actividad 1

- Crea un formulario para entrar en un lugar secreto llamado: FBI .
- Añade los campos adecuados y valídalos.
- **Nombre:** Obligatorio, solo texto.
- **Contraseña:** Obligatoria.(mínimo 8 caracteres, numero, letra, símbolo)

Actividad 2

- Crea un formulario para publicar un producto de segunda mano.
- Añade los campos adecuados y válídalos.
- **Nombre:** Obligatorio, solo texto.
- **E-mail:** Obligatoria, formato correcto.
- **Precio:** Solo número.
- **Aceptar políticas:** Obligatorio.
- Añade otro campo: **Imagen**. Debe ser una imagen que no supere los 5Mb.